

R V S

RVS EDUCATIONAL TRUST'S GROUP OF INSTITUTIONS

RVS Nagar, Dindigul- 624 005.



NM1122 - Node JS, REST API

RECORD NOTE BOOK

Name :

Register No:

Year & Sec:

R V S

RVS EDUCATIONAL TRUST'S GROUP OF INSTITUTION

RVS Nagar, Dindigul- 624 005.



RECORD NOTE BOOK

Reg.No.

*Certify that this is the bonafide record of work done by
Selvan/Selvi.....of the semester
.....Branch during the year 20....-
20.... in the.....Laboratory.*

Staff-in-Charge

Head of the Department

Submitted for the University Practical Examination
on.....2025.

Internal Examiner

External Examine



**9214 - RVS SCHOOL OF ENGINEERING AND
TECHNOLOGY**

COMPUTER SCIENCE AND ENGINEERING

RESTFUL CONTACT MANAGEMENT API

**STUDENT NM ID:
E2224445B5A72AFBFD34DECBDE9008FE**

SUBMITTED BY

VIDHYAVARSHINI S

921423104051

Phase 1 –

Problem Understanding & Requirements (Contact Management

API)

1. Problem Statement

People often struggle to manage and organize their personal and professional contacts effectively. Existing

Solutions may be too complex or lack customization.

A simple, secure, and scalable Contact Management REST API will help users create, read, update, and

Delete (CRUD) contact details easily while supporting future integrations with web or mobile apps.

2. Users & Stakeholders

- End Users: Individuals or organizations who want to manage contact details.
- Admin: Manages user accounts and ensures system reliability.
- Developers: Developers integrating the API into apps (frontend, mobile).
- Stakeholders: College (IBM NM project requirement), project mentors, and team members.

3. User Stories

- As a user, I want to add new contacts with details like name, phone, and email.
- As a user, I want to view all my contacts so that I can see my list easily.
- As a user, I want to search a contact by name/email so I can find information quickly.
- As a user, I want to update a contact's details if they change.
- As a user, I want to delete a contact I no longer need.
- As an admin, I want to manage users (register/login).

4. MVP Features

- User Authentication (Sign up, Login, Token-based auth).
- CRUD operations for contacts:

O Create Contact

O Read Contact(s)

O Update Contact

O Delete Contact

- Search/filter contacts by name/email.

5. Wireframes / API Endpoint List

(API Endpoint examples with REST principles)

Auth

- POST /api/auth/register → Register a new user
- POST /api/auth/login → Login user

Contacts

- POST /api/contacts → Create a contact
- GET /api/contacts → Get all contacts
- GET /api/contacts/:id → Get contact by ID
- PUT /api/contacts/:id → Update contact by ID
- DELETE /api/contacts/:id → Delete contact by ID

Phase 2 —

Solution Design & Architecture

Project: RESTful Contact Management API

Goal: Define the technical foundation and high-level structure of the application before coding begins.

1. Tech Stack Selection

Choose the core technologies that will power the API and its clients.

Example choices:

- Backend: Node.js with Express or Fastify for REST endpoints
 - Database: PostgreSQL (relational) or MongoDB (document)
 - ORM / Query Layer: Prisma or Sequelize
 - Authentication: JWT-based auth (or OAuth2 if third-party access is needed)
 - Infrastructure & Tools: Docker for containerization, GitHub Actions for CI/CD, hosting on AWS/GCP/Azure.
- Purpose: Ensures everyone agrees on the runtime environment and tools early, avoiding late rework.

2. UI Structure / API Schema Design

Even though this is an API-first project, you still outline:

- UI Structure (if a web or admin console is planned): High-level wireframes showing how users create, view, and

Manage contacts.

- API Schema: Define the REST resources and JSON structure—for example, a Contact object with id, name, email,

Phone, timestamps, etc.

Deliverable: An OpenAPI/Swagger specification documenting endpoints like

GET /api/v1/contacts, POST /api/v1/contacts, PATCH /api/v1/contacts/{id}, etc.

3. Data Handling Approach

Decide how the system stores, validates, and protects data:

- Database schema (tables/collections, indexes, relationships)
- Validation rules (e.g., email format, unique phone numbers)
- CRUD operations and how they map to HTTP methods
- Pagination, filtering, and search strategy
- Backup and recovery plans, encryption at rest/in transit.

4. Component / Module Diagram

Create a high-level architecture diagram showing all major parts and how they interact.

Typical modules:

- API Gateway / Router – handles incoming HTTP requests
- Business Logic Layer – services for contact creation, updates, etc.
- Data Access Layer – ORM or query builder talking to the database
- Authentication & Authorization Module – login, token verification
- Notification/Email Service – if you send confirmation mails or share contacts.

This diagram clarifies boundaries and responsibilities before development starts.

5. Basic Flow Diagram

Depict the end-to-end request flow:

1. Client sends a request (e.g., Create Contact)

2. API Gateway authenticates and routes it

3. Business logic validates and processes

4. Data layer performs DB operations

5. Response returned to the client.

Including error paths (e.g., invalid token) helps everyone understand runtime behavior.

Why This Phase Matters

Completing these tasks by Week 7 ensures the RESTful Contact Management API is well-planned, consistent,

And scalable. It provides a single reference for developers, testers, and stakeholders before moving into detailed

Implementation and coding.

Phase 3:

RESTful Contact Management API

•

1. Project Setup

Choose language & framework (e.g., Node.js + Express, Python + Flask/FastAPI, Java + Spring

Boot).

Initialize the project (npm init, pip install, etc.).

Install required dependencies (web framework, body parser, database driver, testing framework).

Setup folder structure (e.g., controllers/, models/, routes/).

2. Core Features Implementation

RESTful endpoints for managing contacts:

GET /contacts → Fetch all contacts

GET /contacts/{id} → Fetch single contact

POST /contacts → Create new contact

PUT /contacts/{id} → Update existing contact

DELETE /contacts/{id} → Remove contact

Contact model fields (example):

```
{
  "id": 1,
  "name": "Alice Johnson",
  "email": "alice@example.com"
  "phone": "+1234567890"
```

```
}
```

3. Data Storage

Option 1 (Local State): Store in memory (array/object) for quick prototyping.

Option 2 (Database):

SQL (PostgreSQL, MySQL, SQLite) with ORM (Sequelize, SQLAlchemy, Hibernate).

NoSQL (MongoDB, DynamoDB) for flexible schemas.

4. Testing Core Features

Unit tests for controllers/services.

API testing using Postman, curl, or automated tests with Jest, Pytest, or Junit.

Example test: check that POST /contacts creates a contact and GET /contacts returns it.

6. Version Control (GitHub)

Initialize git repo (git init).

Push to GitHub (git remote add origin ...).

Use feature branches (feature/add-contact, bugfix/update-contact).
Add CI/CD (optional) for automated testing.

Phase 4:

Enhancement (Design & Development)

Enhancements focus on improving the API's functionality, performance, security, and developer

Experience to make it robust and scalable.

1. Functionality and Resource Design

Area

Enhancement Details

Data Access

Implement Filtering, Sorting, and Pagination

(e.g., GET

/contacts?city=London&sort=-lastName&page=

1&limit=20) to handle large datasets efficiently.

Updates

Support the PATCH method for partial

Updates (modifying only specific fields) to

Conserve bandwidth.

Relationships

Design endpoints for Nested Resources to

Clearly represent relationships (e.g.,

/contacts/{contactId}/addresses).

Bulk Operations

Provide endpoints for batch processing (e.g.,

POST /contacts/bulk) to create or update

Multiple contacts in one request.

Data Customization

Allow clients to request only necessary fields

(Field Selection) to reduce payload size (e.g.,

?fields=firstName,email).

2. Security and Compliance

Area

Enhancement Details

Data Security

Enforce HTTPS/SSL for all traffic to ensure

Data is encrypted in transit.

Access Control

Implement secure authentication using OAuth

2.0 or JWT (JSON Web Tokens), coupled with

2.1

Role-Based Access Control (RBAC).

Abuse Prevention

Implement Rate Limiting to protect against

Area

Enhancement Details

Brute-force attacks and denial-of-service by

Restricting request volume per client

Input Integrity

Validate and Sanitize all incoming user input

Rigorously to prevent common vulnerabilities

Like SQL injection.

3. Developer Experience

- API Versioning: Implement versioning (e.g., /v2/contacts) to allow the introduction of nonbackward-compatible changes without disrupting existing clients.

- Documentation: Generate and maintain interactive documentation using OpenAPI

(Swagger) to detail every endpoint and data model.

- Consistent Error Handling: Use standard HTTP Status Codes consistently (e.g., 201

Created, 400 Bad Request, 404 Not Found) and provide clear, uniform, machine-readable error bodies.

Deployment (Infrastructure & Operations)

Deployment focuses on making the API available securely, reliably, and with high availability.

1. Deployment Architecture

Component

Purpose & Recommended Tools

Containerization

Package the API and all its dependencies into

Docker images for environment consistency.

Orchestration

Manage, scale, and auto-heal containers using

Kubernetes (K8s) or cloud services like AWS

ECS/Azure AKS.

API Gateway

A centralized entry point for managing security,

Rate limiting, and traffic routing across multiple

Services (AWS API Gateway, Azure API

Management, Kong).

Database

Use a scalable, managed database service

(e.g., AWS RDS, Google Cloud SQL) for

Reliability and simplified operations.

2. CI/CD Pipeline

- Continuous Integration (CI): Automate code building, running Unit Tests/Integration Tests, and creating the Docker image upon every code commit.

- Continuous Deployment (CD): Automate the deployment of the new, tested Docker Image to the production environment, minimizing manual intervention and speeding up Releases.

3. Advanced Deployment Strategies

- Blue/Green Deployment: Maintain two identical environments (Blue is live, Green is New). Traffic is switched instantly to the new version once verified, allowing for instant Rollback

- Canary Deployment: Roll out the new version to a small percentage of users first. Monitor Its performance and, if stable, gradually increase traffic to the new version.

4. Monitoring, Logging, and Alerting

- Logging: Centralize logs using a solution like the ELK Stack (Elasticsearch, Logstash, Kibana) or cloud logging tools for easy debugging and analysis.

- Metrics & Monitoring: Track key performance indicators (KPIs) like latency, error rates (5xx, 4xx), and request throughput. Use tools like Prometheus and Grafana to

Visualize metrics and set up automated alerts for critical failures.

- Health Checks: Implement dedicated health endpoints (e.g., /health) that load balancers

And orchestrators can use to check the API's status and automatically route traffic away

From unhealthy instances.

Phase 5 :

FINAL WALKTHROUGH – RESTful Contact Management API

 Project Title:

RESTful Contact Management API

 Objective:

The main goal of this project is to develop a RESTful API that can manage contact data (like name, email,

Phone) using CRUD operations — Create, Read, Update, and Delete.

It shows how frontend (HTML, CSS, JS) and backend (Node.js + Express) work together through RESTful

HTTP methods.

 Working Explanation (Final Overview):

1. Server Setup (Backend) o A Node.js server was created using Express.js framework.
O

The server runs on port 5000 and provides RESTful endpoints to handle data. O

The API handles data using a temporary in-memory JSON array (no database).

The backend supports these operations:

O GET /api/contacts → Fetch all contacts o

POST /api/contacts → Add a new contact o PUT

/api/contacts/:id → Update existing contact o

DELETE /api/contacts/:id → Delete contact by

ID

2. Frontend Integration (Client Side) o Inside the public/ folder, three files (index.html,

Style.css, script.js) make the front interface.

O The HTML file contains a simple form with input fields for Name, Email, and Phone.

O When you click “Add Contact”, it calls the backend API using Fetch API (JavaScript).

O The contact list dynamically displays all saved contacts using the GET API. O Each Contact has a Delete button that triggers a DELETE API call.

3. How the System Works (Step-by-Step):

O Start the server using node server.js. o

Visit <http://localhost:5000> in

Your browser.

O Fill in the contact form → Name, Email, Phone → click “Add Contact”.

O The new contact gets added via POST request and shows instantly on the Page. O

You can delete a contact using the delete button (DELETE

Request). O All data is fetched in real-time using Fetch API (GET request).

4. Demonstration Summary: Frontend: Displays all contacts and provides form input.

 Backend: Processes API requests and responses.

 Communication: RESTful architecture using HTTP methods.

 Data Flow: User → Frontend → API → Data → Response → Frontend Update.

 Final Result:

The project successfully demonstrates a fully working RESTful Contact Management System where users can:

- Add new contacts
- View existing contacts
- Delete unnecessary contacts

The app proves how RESTful APIs work in real-time with frontend and backend integration using Node.js + Express

+ HTML + JS.

 Future Enhancements:

- Use MongoDB to permanently store contacts.
- Add Search and Edit features.
- Implement User Authentication (Login/Register).
- Convert the frontend to a React app later.

SOURCE CODE

 server.js

```

Const express = require('express');

Const cors = require('cors'); const

App = express();

App.use(cors());

App.use(express.json());

App.use(express.static('public'));

Let contacts = [

  { id: 1, name: "Vidhya", email: vidhya@gmail.com, phone: "9876543210" },

  { id: 2, name: "Karthik", email: karthik@gmail.com, phone: "9876500000" }

];

// Get all contacts

App.get('/api/contacts', (req, res) => {

  Res.json(contacts);

});

// Add new contact
App.post('/api/contacts', (req, res) => {

  Const newContact = { id: Date.now(),

  Name: req.body.name, email:

  Req.body.email, phone: req.body.phone

  };

  Contacts.push(newContact);

  Res.json({ message: 'Contact added successfully', contact: newContact });

```

```

});

// Update contact

App.put('/api/contacts/:id', (req, res) => {

  Const contactId = parseInt(req.params.id);

  Const index = contacts.findIndex(c => c.id === contactId);

  If (index !== -1) {

    Contacts[index] = { ...contacts[index], ...req.body };

    Res.json({ message: 'Contact updated', contact: contacts[index] });

  } else {

    Res.status(404).json({ message: 'Contact not found' });

  }

});

// Delete contact

App.delete('/api/contacts/:id', (req, res) => { const

  contactId = parseInt(req.params.id); contacts =

  contacts.filter(c => c.id !== contactId); res.json({

  message: 'Contact deleted successfully' });

});

Const PORT = 5000;

App.listen(PORT, () => console.log(` Server running on http://localhost:\${PORT}`));

📁 public/index.html

<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>RESTful Contact Management API</title>

```

```

<link rel="stylesheet" href="style.css" />

</head>

<body>

<h1>Contact Management System</h1>

<div class="form-container">

<input type="text" id="name" placeholder="Enter Name" />

<input type="email" id="email" placeholder="Enter Email" />

<input type="text" id="phone" placeholder="Enter Phone" />

<button onclick="addContact()">Add Contact</button>

</div>

<h2>Contact List</h2>

<ul id="contactList"></ul>

<script src="script.js"></script>

</body>
</html>

```

 public/script.js

```

Const apiURL = '/api/contacts';

Async function fetchContacts() {

Const res = await fetch(apiURL);

Const data = await res.json();

Const list = document.getElementById('contactList');

List.innerHTML = ''; data.forEach(contact => {

Const li = document.createElement('li');

li.innerHTML = `

<strong>${contact.name}</strong> - ${contact.email} - ${contact.phone}

<button onclick="deleteContact(${contact.id})">Delete</button>

`;

List.appendChild(li);

```



```

});

}

Async function addContact() {

  Const name = document.getElementById('name').value;

  Const email = document.getElementById('email').value;

  Const phone = document.getElementById('phone').value;

  If (!name || !email || !phone) {

  Alert('Please fill all fields'); return;

  }

  Await fetch(apiURL, {

  Method: 'POST',

  Headers: { 'Content-Type': 'application/json' },

  Body: JSON.stringify({ name, email, phone })

  });

  Document.getElementById('name').value = '';

  Document.getElementById('email').value = '';

  Document.getElementById('phone').value = '';

  fetchContacts();

}

Async function deleteContact(id) {

  Await fetch(`${apiURL}/${id}`, { method: 'DELETE' });

  fetchContacts();

}

fetchContacts();

📁 public/style.css

Body {

  Font-family: Arial, sans-serif;

  Text-align: center; background-color:

```

```
#f4f4f4;
```

```
}
```

H1

```
{
```

```
Color: #2d6cdf;
```

```
}
```

```
.form-container {
```

```
Margin: 20px;
```

```
}
```

```
Input { padding: 8px;
```

```
Margin: 5px; border:
```

```
1px solid #ccc; borderradius: 5px;
```

```
}
```

Button

```
{
```

```
Padding: 8px 12px;
```

```
Background-color: #2d6cdf;
```

```
Color: white; border:
```

```
None; border-radius: 5px;
```

```
Cursor: pointer;
```

```
}
```

```
Button:hover {
```

```
Background-color: #1a4cad;
```

```
}
```

```
UI
```

```
{
```

```
List-style: none;
```

```
Padding: 0;
```

```
}
```

```
Li
```

```
{
```

```
Background: #fff;
```

```
Margin: 10px auto;
```

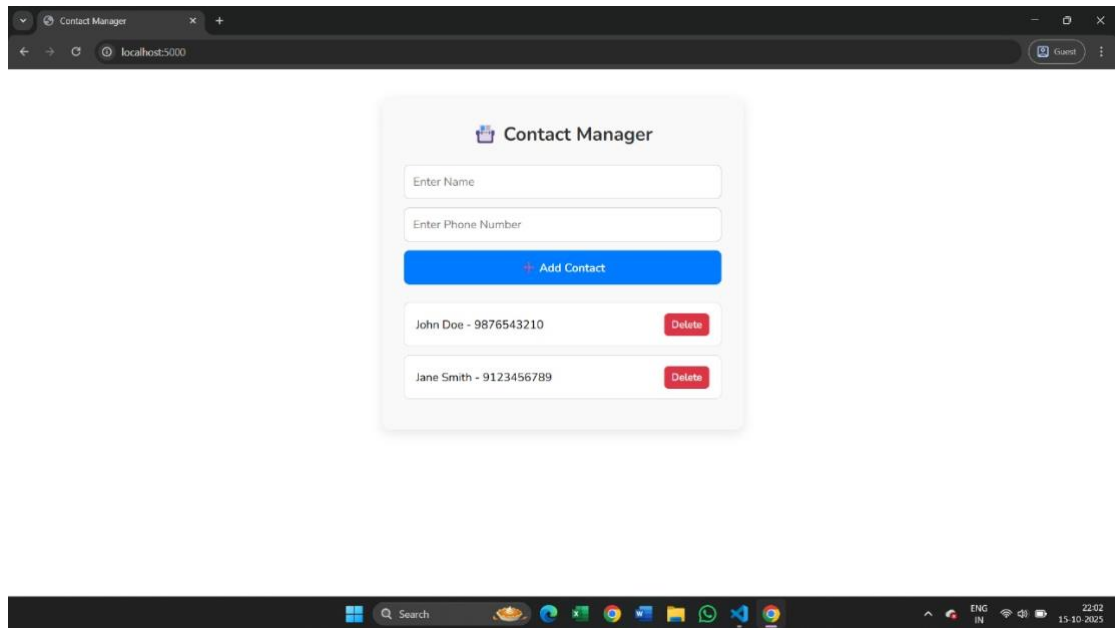
```
Padding: 10px; width:
```

```
300px; border-radius:
```

```
5px; box-shadow: 0 0 5px
```

```
#ccc; }
```

Screenshot:



GitHub Repo Link:

<https://github.com/vidhyasakthi>

mohan/Restful-contactmanagement-API-.git

